

An introduction to Python language

Constructor

Dr. Ahmed H. Aliwy

University of Kufa

2021

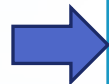
Python basics

- String
- Lists
- Tuples
- Dictionaries

Strings

- ❑ Text is represented in programs by the *string* data type.
- ❑ A string is a sequence of characters enclosed within quotation marks (") or apostrophes (').

```
str1="Hello"  
str2='spam'  
print(str1, str2)  
print(type(str1))  
print(type(str2))
```



```
output  
Hello spam  
<class 'str'>  
<class 'str'>
```

- ❑ Getting the length of a string using len(string)

```
str1="Hello"  
y= len(str1)  
print(y)
```



```
output  
5
```

Strings

□ Getting a character from string.

H	e	l	l	o		B	o	b
0	1	2	3	4	5	6	7	8
-9	-8	-7	-6	-5	-4	-3	-2	-1

```
greet = "Hello Bob"
```

```
print(greet[0])
```

```
print(greet[0], greet[2], greet[4])
```

```
x = 8
```

```
print(greet[x - 2])
```

```
print(greet[-1])
```

```
print(greet[-3])
```

H

H l o

B

b

B

```
print(greet[0:3])
```

```
print(greet[5:9])
```

```
print(greet[:5])
```

```
print(greet[5:])
```

```
print(greet[:])
```

Hel

Bob

Hello

Bob

Hello Bob

Strings

- ❑ Concatenation “glues” two strings together (+)
- ❑ Repetition builds up a string by multiple concatenations of a string with itself (*)
- ❑ The function `len()` will return the length of a string.

[illegible]

Strings

- ❑ *Changing the specific letters in a string is limited!*

```
>>> x="spam"
```

```
>>> x[1]='d'
```

Traceback (most recent call last):

File "<pyshell#29>", line 1, in <module>

x[1]='d'

TypeError: 'str' object does not support item assignment

```
>>>
```

- ❑ *Instead we use*

```
>>> x='d'+x[1:]
```

```
>>> x
```

```
'dpam'
```

```
>>>
```

• *Note: `ord("A")=>65` and `chr(65)=>'A'`*

Strings

- * Python's triple quotes allow strings to span multiple lines, including NEWLINEs, TABs, and any other special characters.

```
"""This is  
a legal String."  
"and this".""""
```

Note: we can use r “anystring” which equivalent to @“anystring” in C# language

- * A string can represent characters by preceding them with a backslash.

- * \t tab character
- * \n new line character
- * \" quotation mark character
- * \\ backslash character

```
* ..... >>> print"Hello\tthere\nHow are you?"  
Hello   there  
How are you?  
>>>  
>>>
```

Some String functions

- * `count(str, beg= 0,end=len(string))` Counts how many times str occurs in string or in a substring of string if starting index beg and ending index end are given
- * `endswith(suffix, beg=0, end=len(string))` Determines if string or a substring of string (if starting index beg and ending index end are given) ends with suffix; returns true if so and false otherwise
- * `find(str, beg=0, end=len(string))` Determine if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise
- * `index(str, beg=0, end=len(string))` Same as find(), but raises an exception if str not found
- * `isalnum(), isalpha(), isdigit()...` return true according to conditions.
- * `len(string)` Returns the length of the string

Some String functions

- * lstrip(), rstrip(), strip() are ways to remove whitespace or selected characters
- * replace(old, new [, max]) Replaces all occurrences of old in string with new or at most max occurrences if max given
- * split(str="", num=string.count(str)) Splits string according to delimiter str (space if not provided) and returns list of substrings; split into at most num substrings if given
- * startswith(str, beg=0, end=len(string)) Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise
- * ... And many more.

Lists Data structure

- Strings are always sequences of characters, but *lists* can be sequences of arbitrary values.

- Lists can have numbers, strings, or both!

```
myList = [1, "Spam ", 4, "U"]
```

- Lists are *mutable*, meaning they can be changed. Strings can **not** be changed.

- myList = [34, 26, 15, 10]

- myList[2]=5 → myList is [34, 26, 5, 10]

Lists Data structure

- Producing a list

- `X=[]` # empty List
 - `Y= [1,2,3,4,5,6,7,8,9]`
 - `Y = [a for a in range(1,10)]`
 - `Z= [1,2,[5,8],(1,5),(9,3,5)]`
- the same
- 

Lists Data structure

mylist=["a","b",3.58,"d",4,0]		
mylist[0] mylist[2]	a 3.58	Indexing
mylist[-1] mylist[-2]	0 4	
mylist[1:4]	["b",3.58,"d"]	Slicing (like strings)
"b" in mylist "e" not in mylist	1 or True 1 or True	Check existing an item
mylist.append(8)	["a","b",3.58,"d",4,0,8]	
Add to end of list		

Lists Data structure

We can convert any sequence to list by using `list(seq)`:

EX: `L=list("ahmed")`  `['a', 'h', 'm', 'e', 'd']`

SN	Methods with Description
1	<code>list.append(obj)</code> Appends object obj to list
2	<code>list.count(obj)</code> Returns count of how many times obj occurs in list
3	<code>list.extend(seq)</code> Appends the contents of seq to list
4	<code>list.index(obj)</code> Returns the lowest index in list that obj appears
5	<code>list.insert(index, obj)</code> Inserts object obj into list at offset index
6	<code>list.pop(obj=list[-1])</code> Removes and returns last object or obj from list
7	<code>list.remove(obj)</code> Removes object obj from list
8	<code>list.reverse()</code> Reverses objects of list in place
9	<code>list.sort([func])</code> Sorts objects of list, use compare func if given

Lists Data structure : examples

```
>>> L1=['as',13,'df']
>>> L2=[15,'aas','df']
>>> L1.append('ali')
>>> L1
['as', 13, 'df', 'ali']
>>> L2.count('as')
0
>>> L1.count('as')
1
>>> L2.extend(L1)
>>> L2
[15, 'aas', 'df', 'as', 13, 'df', 'ali']
>>> L2.index('as')
3
>>> L1.insert(2,'zaid')
>>> L1
['as', 13, 'zaid', 'df', 'ali']
>>> L1.pop()
'ali'
>>> L1
['as', 13, 'zaid', 'df']
>>> L2.remove('as')
>>> L2
[15, 'aas', 'df', 13, 'df', 'ali']
>>> L2.reverse()
>>> L2
['ali', 'df', 13, 'df', 'aas', 15]
>>> L2.sort()
>>> L2
[13, 15, 'aas', 'ali', 'df', 'df']
```

Lists Data structure

- Nested list

الكود البرمجي

```
a=[[1,2,3],[4,5,6],[7,8,9]]  
print(a[0][0])  
print(a[2][0])
```

الايخراج

1
7

Tuples Data structure

- Tuples - sequence of values like lists, but cannot be changed after it is created.
- The position in tuple has semantic comparing with lists.

```
>>> mytuple=(2,3,4,5)
```

```
>>> mytuple[0]
```

```
2
```

```
>>> mytuple[0]=9
```

Traceback (most recent call last):

File "<pyshell#82>", line 1, in <module>

mytuple[0]=9

TypeError: 'tuple' object does not support item assignment

- We can write List of tuples: `LT=[('A',65),('B',66)]`
- `tuple(seq)`: Converts a list into tuple.

Dictionaries

- Dictionaries store a *mapping* between a set of keys and a set of values
 - Keys can be any *immutable* type.
 - Values can be any type
 - A single dictionary can store values of different types
- You can define, modify, view, lookup or delete the key-value pairs in the dictionary
- Python's dictionaries are also known as *hash tables* and *associative arrays*
- no duplicate key is allowed

Dictionaries

```
dict = {'Alice': '2341', 'Beth': '9102', 'Cecil': '3258'}
```

```
dict1 = { 'abc': 456 };
```

```
dict2 = { 'abc': 123, 98.6: 37 };
```

- To access dictionary elements:
 - *dict_name[key]=> ex: dict2['abc']=> 123*
- Updating Dictionary:
 - *dict2['abc']=453*
- Delete Dictionary Elements:
 - `del dict['Alice']; # remove entry with key Alice'`
 - `dict.clear(); # remove all entries in dict`
 - `del dict ; # delete entire dictionary`

Dictionary methods

Methods with Description

`dict.clear()`

Removes all elements of dictionary *dict*

`dict.copy()`

Returns a shallow copy of dictionary *dict*

`dict.get(key, default=None)`

For *key* key, returns value or default if key not in dictionary

`dict.items()`

Returns a list of *dict*'s (key, value) tuple pairs

`dict.keys()`

Returns list of dictionary *dict*'s keys

`dict.values()`

Returns list of dictionary *dict*'s values

Dictionary methods

```
>>> mydic={'a':10,'b':11,'c':12,'d':13}
```

```
>>> x=mydic.copy()
```

```
>>> print(x)
```

```
{'a': 10, 'd': 13, 'c': 12, 'b': 11}
```

```
>>> y=mydic.get('m')
```

```
>>> print(y)
```

```
None
```

```
>>> y=mydic.get('a')
```

```
>>> print(y)
```

```
10
```

```
>>> mydic.values()
```

```
dict_values([10, 13, 12, 11])
```

```
>>> mydic.items()
```

```
dict_items([('a', 10), ('d', 13), ('c', 12), ('b', 11)])
```

```
>>>
```